

Documento 04 — Plan y evidencias de Pruebas

Proyecto: Sistema Inteligente de Gestión y Análisis Documental **Versión:** 1.0 **Fecha:** 8 de septiembre de 2026 **Estado:** Final

1. Estrategia de pruebas

1.1 Enfoque

Se aplica un enfoque de pruebas **funcionales de caja negra** sobre la aplicación desplegada en producción (Vercel + Render), verificando que cada requerimiento funcional del Documento 01 se cumple correctamente desde la interfaz de usuario y la API REST.

1.2 Niveles de prueba

Nivel	Descripción	Herramienta
Unitaria	Scripts de validación del backend (pipeline de IA, extracción de texto, embeddings)	Scripts en 10 - ../scripts/
Integración	Verificación de endpoints REST (auth, repos, files, search)	smokeTest.js , curl/Postman
E2E / Funcional	Flujo completo desde la interfaz web en el navegador	Prueba manual en la app desplegada
API de IA	Validación del pipeline de clasificación, resumen, extracción y embeddings	testOpenRouter.js , testPipelineMock.js

1.3 Ambiente de pruebas

Componente	Entorno	URL
Frontend	Vercel (producción)	https://sistema-documental-ia.vercel.app
Backend	Render (producción)	https://sistema-documental-ia.onrender.com
Base de datos	MongoDB Atlas (cluster compartido)	clusterdocia.xhhvagr.mongodb.net
IA	OpenRouter (DeepSeek + OpenAI embeddings)	https://openrouter.ai/api/v1

1.4 Datos de prueba

- **Usuario de prueba:** prueba@docia.com / Test1234!
- **Documento de prueba:** `generateTestPdf.js` genera un PDF mínimo con texto en Helvetica; también se prueban archivos TXT y DOCX reales.
- **Repositorio de prueba:** creado durante la ejecución de los casos de prueba.

2. Casos de prueba

CP-01: Registro de usuario nuevo

Campo	Detalle
ID	CP-01
Requerimiento	RF01
Precondición	No haber registrado previamente el correo
Pasos	1. Abrir <code>/register</code> · 2. Completar nombre, correo, contraseña (≥ 8 caracteres) · 3. Pulsar "Registrarse"
Resultado esperado	Se crea la cuenta, se autentica automáticamente y se redirige al dashboard
Estado	Aprobado
Evidencia	Login posterior con las mismas credenciales devuelve token y datos del usuario (<code>POST /api/auth/register</code> → 201 con token)

CP-02: Inicio de sesión con credenciales válidas

Campo	Detalle
ID	CP-02
Requerimiento	RF02, RF03
Precondición	Usuario registrado
Pasos	1. Abrir <code>/login</code> · 2. Ingresar correo y contraseña correctos · 3. Pulsar "Iniciar sesión"
Resultado esperado	Se obtiene token JWT y se accede al dashboard
Estado	Aprobado

Campo	Detalle
Evidencia	POST /api/auth/login con prueba@docia.com → 200 con token JWT y datos; rutas protegidas accesibles con el token

CP-03: Inicio de sesión con credenciales inválidas

Campo	Detalle
ID	CP-03
Requerimiento	RF02, RF20
Precondición	—
Pasos	1. Abrir /login · 2. Ingresar correo o contraseña incorrectos · 3. Pulsar "Iniciar sesión"
Resultado esperado	Se muestra mensaje de error "Correo o contraseña inválidos"
Estado	Aprobado
Evidencia	POST /api/auth/login con contraseña incorrecta → 401 con mensaje Correo o contraseña inválidos

CP-04: Crear repositorio

Campo	Detalle
ID	CP-04
Requerimiento	RF04
Precondición	Usuario autenticado
Pasos	1. Navegar a /repos · 2. Pulsar "Nuevo repositorio" · 3. Ingresar nombre y descripción · 4. Guardar
Resultado esperado	El repositorio aparece en la lista con su nombre y descripción
Estado	Aprobado
Evidencia	POST /api/repos → 201 con el objeto creado; GET /api/repos lo lista con conteo de documentos

CP-05: Subir documento (PDF)

Campo	Detalle
ID	CP-05
Requerimiento	RF05, RF06
Precondición	Repositorio existente
Pasos	1. Abrir detalle del repositorio · 2. Seleccionar archivo PDF · 3. Subir
Resultado esperado	El documento aparece en la lista con estado "pendiente" y luego "processing"
Estado	Aprobado
Evidencia	POST /api/files/upload (multipart) → 201 con documento; GET /api/files lo muestra con status: "pending" que cambia a "completed" tras procesamiento

CP-06: Rechazo de archivo con extensión no permitida

Campo	Detalle
ID	CP-06
Requerimiento	RF06
Precondición	Repositorio existente
Pasos	1. Intentar subir un archivo .exe o .jpg
Resultado esperado	El sistema rechaza el archivo con mensaje claro
Estado	Aprobado
Evidencia	POST /api/files/upload con extensión no válida → 400 "Tipo de archivo no permitido"

CP-07: Ver resultado del procesamiento IA (clasificación, resumen, datos extraídos)

Campo	Detalle
ID	CP-07
Requerimiento	RF08, RF09, RF10, RF11, RF12
Precondición	Documento procesado con estado "completed"

Campo	Detalle
Pasos	1. Abrir detalle del documento · 2. Revisar categoría, resumen y campos extraídos
Resultado esperado	Se muestra la categoría asignada (Contrato/Factura/Reporte/Otro), un resumen de 3-5 líneas, y campos clave (fechas, montos, partes)
Estado	Aprobado
Evidencia	<code>GET /api/files/:id → 200 con category , summary , extractedInfo , textContent y chunks ; verificación con testPipelineMock.js muestra campos correctos en la BD</code>

CP-08: Búsqueda semántica

Campo	Detalle
ID	CP-08
Requerimiento	RF13, RF14
Precondición	Al menos un documento procesado con embeddings
Pasos	1. Navegar a <code>/search</code> · 2. Escribir una consulta semántica (ej. "contrato de servicios") · 3. Pulsar buscar
Resultado esperado	Se muestran fragmentos relevantes de los documentos con su fuente
Estado	Aprobado
Evidencia	<code>POST /api/search/search con query → 200 con array de resultados que incluyen contenido, nombre del documento, categoría y score de similitud; verificación con checkVectorIndex.js confirma índice vector_index activo</code>

CP-09: Chat RAG (pregunta en lenguaje natural)

Campo	Detalle
ID	CP-09
Requerimiento	RF15, RF16
Precondición	Documentos procesados con embeddings
Pasos	1. Navegar a <code>/search</code> · 2. Escribir una pregunta como "¿Cuál es el monto del contrato?" · 3. Enviar

Campo	Detalle
Resultado esperado	El sistema responde con información basada en los documentos y cita las fuentes
Estado	Aprobado
Evidencia	POST /api/search/chat con question → 200 con answer (respuesta de DeepSeek) y sources (array de documentos con nombre, categoría y fragmento); verificación con smokeTest.js

CP-10: Eliminar documento

Campo	Detalle
ID	CP-10
Requerimiento	RF07, RF23
Precondición	Documento existente; usuario propietario o admin
Pasos	1. Abrir detalle del documento · 2. Pulsar "Eliminar" · 3. Confirmar
Resultado esperado	El documento desaparece de la lista y el archivo físico se elimina
Estado	Aprobado
Evidencia	DELETE /api/files/:id → 200 "Documento eliminado"; GET /api/files/:id → 404; archivo eliminado de backend/uploads/

3. Resumen de resultados

Caso de prueba	Estado	Observación
CP-01: Registro	Aprobado	Cuenta creada, autenticado
CP-02: Login válido	Aprobado	Token JWT devuelto
CP-03: Login inválido	Aprobado	Mensaje de error claro
CP-04: Crear repositorio	Aprobado	Repo listado con conteo
CP-05: Subir PDF	Aprobado	Procesamiento IA completo
CP-06: Rechazar archivo inválido	Aprobado	Mensaje de error claro
CP-07: Ver análisis IA	Aprobado	Categoría, resumen, datos clave visibles

Caso de prueba	Estado	Observación
CP-08: Búsqueda semántica	Aprobado	Resultados relevantes con fuente
CP-09: Chat RAG	Aprobado	Respuesta basada en documentos con fuentes
CP-10: Eliminar documento	Aprobado	Archivo y registro eliminados

Total: 10/10 aprobados (100%)

4. Registro de defectos

ID	Defecto	Severidad	Estado	Resolución
—	No se identificaron defectos críticos durante las pruebas en producción	—	—	—

Limitaciones conocidas (no son defectos)

Limitación	Descripción	Mitigación
No OCR	Los PDF escaneados sin capa de texto no se pueden procesar	Se marca como error con mensaje claro; se documenta como exclusión
No multi-idioma	La interfaz y los prompts están en español	Se documenta como exclusión en el alcance
Tiempo de procesamiento	Documentos muy extensos pueden tardar más de 60 s en procesarse	Se procesa de forma asíncrona; el usuario ve el progreso
Límite de tamaño	Archivos mayores a 10 MB se rechazan	Configurable con <code>MAX_FILE_SIZE_MB</code>

5. Scripts de automatización de pruebas

Los siguientes scripts se encuentran en la carpeta `10 - Base de datos, scripts o estructura necesaria para reproducir el sistema/`:

Script	Qué valida	Tipo
<code>smokeTest.js</code>	Flujo completo: login → upload → process → resultado	E2E HTTP
<code>testPipelineMock.js</code>	Pipeline de IA completo con mocks (sin llamadas reales)	Integración

Script	Qué valida	Tipo
testOpenRouter.js	Clasificación, resumen, extracción y embeddings contra OpenRouter real	API de IA
testErrorPath.js	Manejo de errores: documento en processing que falla la IA → error controlado	Error path
testRecovery.js	Recuperación de documentos "colgados" en processing	Resiliencia
testDay3.js	Extracción de texto (PDF/TXT) y chunking	Unidades
checkVectorIndex.js	Verifica que el índice <code>vector_index</code> existe en Atlas	Verificación
createVectorIndex.js	Crea el índice vectorial si no existe	Setup
generateTestPdf.js	Genera un PDF mínimo para pruebas	Utilidad
reprocess.js	Reprocesa un documento y sigue el estado hasta completar	Utilidad

6. Conclusiones

- Todos los 10 casos de prueba definidos fueron **aprobados** en el ambiente de producción (Vercel + Render + MongoDB Atlas + OpenRouter).
- No se encontraron defectos críticos durante la fase de pruebas.
- Las limitaciones conocidas (sin OCR, solo español) están documentadas y mitigadas.
- Los scripts de automatización permiten reproducir las pruebas de forma no manual en cualquier momento.

Fin del documento 04.